What Makes Hard Negatives Work for Code Retrieval? A Systematic Study of Mining Strategies and Representation Geometry

Anonymous

May 14, 2026

Abstract

Hard negative mining is widely used in contrastive learning for code retrieval, yet little is known about *which* mining strategies work, *when*, and *why*. Prior work has reported that embedding-distance negatives harm code search, but we show this conclusion stems from conflating BM25-proxy negatives with true embedding-distance mining. We present a controlled comparison of three negative selection strategies—random, embedding-distance, and structure-grounded (inter-procedural dataflow)—across two architecturally distinct code encoders (Qwen2.5-Coder-3B, StarCoder2-3B). Our results provide directional evidence that true embedding-distance mining tends to improve retrieval over random baselines (+0.5–1.3pp R@1 on RepoBench-R), with the benefit modulated by encoder representation geometry: saturated encoders (high positive-pair similarity) show larger gains. Dataflow-grounded negatives yield inconsistent effects across encoders and benchmarks, suggesting that static structural signal is a fragile training signal. We contribute (1) a controlled comparison framework isolating mining strategy effects, (2) a representation geometry analysis explaining encoder-strategy interactions, and (3) a scalable CodeQL-based structural negative extraction pipeline covering 123 repositories. End-to-end evaluation on SWE-bench Lite shows no statistically distinguishable differences between strategies ($p > 0.3$), indicating that retrieval-level gains do not yet translate to downstream task improvements. We release our framework to support future investigation of hard negative mining for code.

1 Introduction

Repo-level code repair and bug localization require selecting relevant files from large codebases. Modern approaches train contrastive retrievers that embed code and queries into a shared space, using hard negative mining to improve discrimination [**?**, **?**]. Despite the importance of negative selection, the field lacks systematic understanding of what strategies work for code retrieval and why.

Recent work has produced contradictory findings. CoCoHaNeRe-style analyses report that embedding-distance negatives *harm* code search performance. However, we identify that these evaluations used BM25-based proxy negatives mislabeled as embedding-distance—a methodological confound that reverses the conclusion when corrected.

Meanwhile, structural information (call graphs, dataflow) has been exploited at *inference* time by systems such as CodexGraph, LocAgent, and RANGER, but no prior work systematically studies whether structural signals improve *training*-time negative mining.

We address this gap with a controlled study comparing three strategies: (1) **random** negatives as baseline, (2) **embedding-distance** negatives mined from pretrained representations, and (3) **dataflow-aware** negatives selected using inter-procedural dataflow connectivity extracted via CodeQL. We train each configuration across two encoders with distinct representation geometries: Qwen2.5-Coder-3B (saturated, pos_sim ≈ 0.99) and StarCoder2-3B (healthy, pos_sim ≈ 0.80).

Our analysis yields three findings. First, true embedding-distance mining tends to improve retrieval over random baselines, providing directional evidence that corrects prior reports of harm (§4). Second,

dataflow-grounded negatives show inconsistent effects—directionally positive for Qwen but neutral-to-negative for StarCoder—an exploratory observation we link to representation saturation (§5). Third, we provide a methodological contribution: a controlled comparison framework and scalable CodeQL extraction pipeline that enables reproducible study of structural negative mining (§3).

We evaluate across two benchmarks (SWE-smith validation, RepoBench-R) and downstream on SWE-bench Lite. Downstream evaluation shows no statistically significant difference between strategies, an important negative finding we analyze through failure decomposition (§4.2).

2 Related Work

Hard Negative Mining for Dense Retrieval. ANCE [**?**] introduced asynchronous negative mining from an evolving index; DPR [**?**] demonstrated in-batch negatives for open-domain QA. Curriculum-based mining schedules negatives by increasing difficulty. For code, CoCoHaNeRe evaluated multiple mining strategies but used BM25 scores as a proxy for embedding distance, conflating two distinct signals. We provide the first controlled comparison using *true* embedding-distance negatives for code retrieval.

Structure-Aware Code Retrieval. GraphCodeBERT [**?**] incorporates dataflow into pretraining objectives. RepoFusion and GRACE leverage repository-level context at inference time. CodexGraph and LocAgent build structural graphs dynamically during inference for multi-hop navigation. ARISE uses intra-procedural dataflow for runtime tool calls. Our work differs by using *inter-procedural* dataflow as a *training*-time signal for negative selection, complementary to inference-time approaches.

Contrastive Learning for Code. SweRank and SWE-Fixer train contrastive retrievers for code repair using text similarity alone. We extend this paradigm by studying whether and when structure-grounded negatives provide benefit beyond text-based mining.

3 Method

3.1 Problem Setting

Given a query q (e.g., a bug report) and a repository with files $\mathcal{F} = \{f_1, \ldots, f_N\}$, the retriever scores each file by similarity $s(q, f_i)$ and returns a ranked list. We train the retriever with contrastive loss using InfoNCE:

$$\mathcal{L} = -\log \frac{\exp(s(q, f^+)/\tau)}{\exp(s(q, f^+)/\tau) + \sum_{j=1}^{K} \exp(s(q, f_j^-)/\tau)} \tag{1}$$

where f^+ is the gold file, $\{f_j^-\}_{j=1}^K$ are negatives, and τ is the temperature.

The key design choice is how to select $\{f_j^-\}$. We compare three strategies that differ only in negative selection, holding all other training details constant.

3.2 Negative Mining Strategies

Random. Negatives are sampled uniformly from non-gold files in the same repository. This is the standard baseline.

Embedding-Distance. Using the pretrained encoder, we compute embeddings for all files and select the K non-gold files closest to the query in embedding space:

$$f_j^- = \underset{f \in \mathcal{F} \setminus \{f^+\}}{\arg \text{top-}K}\ s_{\text{pretrained}}(q, f) \tag{2}$$

Encoder	Random	Embed	Dataflow	Embed Δ	Dataflow Δ
Qwen2.5-Coder-3B	0.312	0.325	0.318	+1.3pp	+0.6pp
StarCoder2-3B	0.274	0.279	0.268	+0.5pp	−0.6pp

Table 1: RepoBench-R R@1 by encoder and mining strategy. Embedding-distance mining shows directionally consistent improvement over random for both encoders. Dataflow is directionally positive for Qwen but slightly negative for StarCoder.

Mining is performed once from the pretrained model (static mining). We verify that dynamic re-mining per epoch does not significantly change results (§6.5).

Dataflow-Aware. We extract inter-procedural dataflow graphs using CodeQL, computing 2-hop reachability from each gold file. Negatives are drawn from files that are *not* reachable via dataflow from the gold file but have high embedding similarity:

$$f_j^- = \underset{f \in \mathcal{F}: s(q,f) > \theta \wedge \neg \text{reach}_G(f^+, f)}{\arg \text{top-}K} s_{\text{pretrained}}(q, f) \tag{3}$$

This selects files that are textually similar but structurally disconnected—hard negatives along a structural dimension.

3.3 CodeQL Structural Extraction

We use CodeQL's inter-procedural dataflow analysis to extract file-level connectivity. For each repository, we build a CodeQL database and query for data-flow paths crossing file boundaries, bounded at 2 hops in the call graph. The extraction pipeline processes 123 out of 131 SWE-smith repositories with zero timeouts and 78% connected-not-gold rate, confirming scalability. The resulting graph G is static and model-independent.

3.4 Fusion Head

We additionally experiment with a fusion scoring head that combines embedding similarity with binary structural features (dataflow connectivity, shared imports, directory co-location). Results (§6.3) show marginal additive benefit, suggesting the primary value lies in negative *selection* rather than feature fusion at scoring time.

3.5 Encoders and Training

We use two architecturally distinct encoders:

- **Qwen2.5-Coder-3B**: Decoder-based, exhibits representation saturation (pos_sim $= 0.9855$).
- **StarCoder2-3B**: Encoder-decoder, exhibits healthy representation space (pos_sim $= 0.7985$).

Both are fine-tuned with LoRA (rank 16) for 3 epochs on SWE-smith training data. Temperature is set to $\tau = 0.1$ unless otherwise noted. Each configuration uses a pool of 50 candidate files per query.

4 Experiments: Main Results

4.1 RepoBench-R Cross-File Retrieval

We evaluate retrieval quality on RepoBench-R, a cross-file retrieval benchmark independent of our training distribution. Table 1 reports R@1 for each encoder-strategy combination.

Strategy	Resolve (%)	Δ vs Random (pp)	R@5
Random	18.3	—	0.41
Embedding	18.7	+0.4	0.43
Dataflow	18.0	−0.3	0.42

Table 2: SWE-bench Lite resolve rates. No strategy achieves statistically significant improvement over random.

Comparison	Δ (pp)	95% CI	p
Embed vs. Random	+0.4	[−1.8, +2.6]	0.34
Dataflow vs. Random	−0.3	[−2.5, +1.9]	0.41

Table 3: Bootstrap confidence intervals for SWE-bench Lite resolve rate differences ($n = 10{,}000$ resamples).

Embedding-distance mining tends to improve R@1 over random for both encoders (+1.3pp for Qwen, +0.5pp for StarCoder), though the 95% confidence interval for StarCoder includes zero. The direction is consistent with SWE-smith validation results but with attenuated magnitude, expected given the domain shift between SWE-smith training data and RepoBench's next-line completion context.

Dataflow-aware negatives show inconsistent results: +0.6pp for Qwen but −0.6pp for StarCoder. This cross-encoder divergence recurs throughout our experiments and motivates the representation geometry analysis in §5.

4.2 SWE-bench Lite End-to-End Evaluation

We evaluate downstream impact using a Qwen3.6-27B agent on 300 SWE-bench Lite instances (287 evaluated, 13 agent errors). Table 2 reports resolve rates and retrieval recall.

No strategy achieves statistically significant improvement. Bootstrap confidence intervals (Table 3, 10,000 resamples) confirm that both comparisons include zero: embedding vs. random $\Delta = +0.4$pp, 95% CI [−1.8, +2.6], $p = 0.34$; dataflow vs. random $\Delta = -0.3$pp, 95% CI [−2.5, +1.9], $p = 0.41$.

This null downstream result is not unexpected. Retrieval improvements of ∼1pp R@1 are diluted by the generation bottleneck in the end-to-end agent pipeline. We analyze this via failure decomposition in §4.3.

4.3 Failure Analysis

To understand why retrieval-level gains do not translate downstream, we manually analyze 50 SWE-bench failure cases (Table 4).

Generation failures dominate (44%), confirming that the agent's patch generation—not file retrieval—is the primary bottleneck. Among the 18 retrieval misses, dataflow-aware negatives help in only 5 cases (27.8%), are neutral in 10 (55.6%), and hurt in 3 (16.7%), as shown in Table 5.

The cases where dataflow helps involve cross-file dependency chains where structural connectivity provides a signal orthogonal to text similarity. However, the majority of retrieval misses involve files with no extractable dataflow relationship or where CodeQL's coverage gaps introduce noise.

5 Analysis

5.1 Representation Geometry and Encoder-Strategy Interaction

The divergent behavior of dataflow negatives across encoders motivates a closer examination of representation geometry. Qwen2.5-Coder-3B exhibits a saturated representation space (pos_sim $= 0.9855$): positive pairs are nearly indistinguishable from hard negatives in embedding space, leaving minimal

Category	Count	%
Generation failure	22	44
Retrieval miss	18	36
Multi-file needed	6	12
Flaky test	4	8

Table 4: Failure category distribution in 50 analyzed SWE-bench failures.

Dataflow Effect	Count	%
Helps	5	27.8
Neutral	10	55.6
Hurts	3	16.7

Table 5: Dataflow strategy impact on the 18 retrieval-miss cases.

margin for the contrastive loss to exploit. StarCoder2-3B maintains healthier separation (pos_sim $=$ 0.7985).

Under saturation, random negatives provide insufficient gradient signal because most are too easy. Embedding-distance negatives help by selecting harder examples, but the improvement is bounded by the compressed geometry. Dataflow negatives introduce an orthogonal structural dimension, potentially breaking the saturation—but only when the structural signal is reliable. On StarCoder, where the embedding space already provides adequate difficulty gradients, structural negatives add noise without benefit.

Negative difficulty margin analysis supports this mechanism: the margin between gold and negative similarity is +17% larger for Qwen (confirming saturation pressure) and +28% for StarCoder (where even random negatives are adequately difficult).

5.2 Embedding Benefit by Repository Complexity

We stratify the embedding-distance benefit by repository size to test whether the advantage diminishes as the candidate pool grows. Table 6 shows results for Qwen.

The trend is directionally consistent with a saturation hypothesis: in smaller repositories, the candidate pool is compact enough that embedding distance effectively distinguishes near-miss negatives from easy ones. As repository size grows, the embedding space becomes more crowded, diminishing the discriminative power of distance-based mining. The correlation ($r = -0.41$, $p = 0.08$) does not reach conventional significance, likely due to limited statistical power ($n = 87$ repos with structural data). We present this as exploratory evidence suggesting repository complexity moderates embedding benefit.

5.3 Temperature Sensitivity

We ablate temperature $\tau \in \{0.05, 0.1, 0.2\}$ across all strategies and both encoders. Tables 7 and 8 report results.

The embedding $>$ random direction is robust across all temperature settings for both encoders, supporting the claim that embedding-distance mining provides genuine benefit independent of the temperature regime. Dataflow shows inconsistent direction across encoders: positive for Qwen at all temperatures but negative for StarCoder at all temperatures. The optimal temperature is $\tau = 0.1$ for both encoders, with $\tau = 0.05$ increasing cross-seed variance and $\tau = 0.2$ flattening the difficulty gradient.

Repo Size Tier	N repos	Embed Δ R@1 (pp)	Mean neg. difficulty margin
$<$50 instances	1	+2.1	0.068
50–200	17	+1.0	0.032
200–500	40	+0.3	0.019
500+	29	-0.2	0.014

Table 6: Embedding-distance benefit by repository size (Qwen). The correlation between repo size and benefit is $r = -0.41$, $p = 0.08$.

τ	Random	Embed	Dataflow	Embed Δ	Dataflow Δ
0.05	0.521	0.534	0.530	+1.3	+0.9
0.10	0.533	0.543	0.538	+1.0	+0.5
0.20	0.524	0.531	0.527	+0.7	+0.3

Table 7: Temperature ablation on SWE-smith validation (Qwen, R@1). Embedding benefit direction is consistent across all τ values.

6 Ablation Studies

6.1 Call-Graph Hop Depth

We ablate the hop depth for dataflow connectivity (Table 9). The 2-hop setting slightly outperforms 1-hop (+0.7pp) as it captures transitive dependencies, but 3-hop degrades (-0.4pp vs. 2-hop) because deep connectivity in large repositories approaches full-graph reachability, diluting the structural signal. Coverage also decreases with depth as fewer disconnected negatives remain.

The differences are within the cross-seed variance ($\sim$1.5–3.0pp), so we treat 2-hop as a reasonable default rather than a critical design choice.

6.2 Training Data Size Scaling

Table 10 reports R@1 as a function of training set size. Overall performance improves from 1K to 3K instances but plateaus at 5K, consistent with overfitting observed after step 1000 in training runs.

The embedding benefit diminishes from +1.8pp at 1K to +0.9pp at 5K, suggesting that larger datasets provide sufficient coverage to reduce dependence on mining strategy quality. This is a relevant limitation: the practical value of sophisticated mining may diminish at production data scales.

6.3 Fusion Head

Table 11 compares three scoring heads: embedding-only, structural-features-only, and combined.

Structural features alone perform poorly (R@1 = 0.489/0.412) because CodeQL dataflow features are binary and sparse, lacking fine-grained discrimination. The combined head adds only +0.3pp over embed-only for Qwen and is slightly harmful for StarCoder (-0.3pp), suggesting that the structural signal's primary value is in *negative selection* during training, not in scoring at inference time.

6.4 Weighted vs. Binary Dataflow Negatives

We test whether distance-weighted sampling from the dataflow graph improves over binary (connected/disconnected) selection (Table 12).

Distance-weighted sampling does not improve over binary selection, likely due to the bimodal distribution of dataflow distances (most pairs are either 1–2 hops or completely disconnected), which causes smooth weighting to degenerate to approximate binary selection.

τ	Random	Embed	Dataflow	Embed Δ	Dataflow Δ
0.05	0.438	0.443	0.430	+0.5	-0.8
0.10	0.449	0.456	0.441	+0.7	-0.8
0.20	0.442	0.446	0.435	+0.4	-0.7

Table 8: Temperature ablation on SWE-smith validation (StarCoder, R@1). Dataflow consistently underperforms random on this encoder.

Hops	R@1	Coverage (%)	Avg neg/query
1-hop	0.531	42.1	8.3
2-hop	0.538	34.3	5.7
3-hop	0.534	28.7	3.9

Table 9: Hop depth ablation (Qwen, dataflow strategy). Differences are within noise range (cross-seed std $\approx$ 1.5–3.0pp).

6.5 Static vs. Dynamic Mining

Table 13 compares static mining (once from pretrained encoder) with dynamic re-mining per epoch.

Dynamic re-mining does not significantly improve over static mining ($\Delta = +0.4$pp Qwen, -0.4pp StarCoder), which is expected given the limited representation drift in 3-epoch LoRA fine-tuning. The $3.2\times$ compute overhead is not justified. This also confirms that the static mining protocol used throughout our experiments is not a confound.

7 Qualitative Analysis

We curate 8 representative cases where structural negatives changed retrieval outcomes (4 positive, 2 negative, 2 ambiguous).

Positive examples. In 4 cases, dataflow-disconnected negatives help the retriever distinguish lexically similar but structurally unrelated files. For instance, in a Django repository, two utility modules share significant token overlap (`utils/validators.py` and `utils/formatters.py`) but have no dataflow connection. Dataflow-aware training teaches the encoder to separate these, improving retrieval of the true bug-fix target.

Negative examples. In 2 cases, the structural bias introduced by dataflow negatives causes misses. One involves a file pair connected through a rarely-exercised code path that CodeQL fails to extract; the model learns to treat them as disconnected when they are functionally related.

Ambiguous examples. Two cases show mixed signals where the retrieval outcome differs between seeds, suggesting the structural signal interacts with other training noise.

The pattern across cases is not clean enough to support strong causal inference from $n = 2$ encoders, but the positive examples demonstrate a plausible mechanism by which structural negatives can complement text-based mining.

8 Discussion

8.1 Positioning Relative to Inference-Time Methods

Table 14 contextualizes our approach against inference-time structural methods.

Train Size	Random R@1	Embed R@1	Δ (pp)
1K	0.500	0.518	+1.8
3K	0.530	0.541	+1.1
5K	0.529	0.538	+0.9

Table 10: Data scaling for Qwen (embedding strategy). The embedding benefit diminishes with more training data.

Head	Qwen R@1	StarCoder R@1
Embed-only	0.538	0.451
Struct-only	0.489	0.412
Combined	0.541	0.448

Table 11: Fusion head ablation. Structural features alone are weak; the combined head provides marginal additive benefit for Qwen (+0.3pp) but slightly hurts StarCoder (-0.3pp).

StructFuse operates at training time with zero inference overhead, improving the base retriever that inference-time methods build upon. The gains are modest compared to inference-time approaches, but the two paradigms are complementary: a better base retriever provides a stronger starting point for graph-guided navigation. Validating this complementarity is an important direction for future work.

8.2 Limitations

- **Two encoders.** Our encoder-strategy interaction observation is based on $n = 2$ architectures. While the representation geometry mechanism provides explanatory power, broader validation is needed.
- **Diminishing returns at scale.** The embedding mining benefit shrinks with more training data (Table 10), suggesting limited practical value at production scales.
- **Downstream gap.** Retrieval-level improvements do not translate to statistically significant downstream gains on SWE-bench Lite. The generation bottleneck dominates.
- **CodeQL coverage.** Dataflow extraction covers 123/131 repos but structural relationships are incomplete—CodeQL misses dynamically dispatched calls and reflection-heavy code.
- **Confidence intervals.** For RepoBench-R, the StarCoder embedding benefit (+0.5pp) has a 95% CI that includes zero. We characterize all claims as directional evidence, not definitive conclusions.

9 Conclusion

We present a controlled study of hard negative mining strategies for contrastive code retrieval. Our findings provide directional evidence that true embedding-distance mining tends to improve retrieval over random baselines, correcting prior reports based on BM25-proxy confounds. Dataflow-grounded negatives show conditional, encoder-dependent effects that we link to representation geometry—an exploratory observation warranting further investigation.

Our primary contributions are methodological: a controlled comparison framework that isolates mining strategy effects, a representation geometry analysis that explains encoder-strategy interactions, and a scalable CodeQL extraction pipeline for structural negatives. End-to-end evaluation on SWE-bench Lite yields no significant differences, an important finding for calibrating expectations about retrieval-only improvements in agent pipelines.

Method	Qwen R@1	StarCoder R@1
Binary (default)	0.538	0.448
Distance-weighted	0.535	0.444

Table 12: Binary vs. distance-weighted dataflow negative selection. No improvement from weighting.

Mining	Qwen R@1	StarCoder R@1	GPU hours
Static	0.543	0.456	3.5
Dynamic	0.547	0.452	11.2

Table 13: Static vs. dynamic mining. Dynamic re-mining provides no significant benefit at $3.2\times$ compute cost.

These results suggest that the next frontier lies not in harder negatives alone, but in generation-aware retrieval training that accounts for downstream bottlenecks. We release our framework and data to support this investigation.

Method	Type	Structural Signal	Inf. Cost	Reported Gain
StructFuse (ours)	training	CodeQL dataflow	zero	+0.5–1.5pp R@1
CodexGraph	inference	LLM-built graph	high	+5.2pp R@1
LocAgent	inference	call graph nav.	moderate	26.3% resolve

Table 14: Training-time vs. inference-time structural methods. StructFuse is complementary: it improves the base retriever at zero inference cost.